Go Basics – Introduzione e Fondamenti

Panoramica del linguaggio Go

Go, spesso indicato anche come **Golang**, è un linguaggio di programmazione open source sviluppato da Google, progettato per combinare l'efficienza e la semplicità del C con la produttività e la sicurezza tipica dei linguaggi moderni.

L'obiettivo principale di Go è **consentire la costruzione di software veloce, affidabile ed efficiente su larga scala**. È un linguaggio fortemente tipizzato, compilato, con **gestione automatica della memoria (garbage collection)** e **supporto nativo alla concorrenza** tramite goroutine e canali.

Caratteristiche fondamentali

- **Facilità d'apprendimento e utilizzo**: la sintassi è minimale e coerente.
- **Concorrenza integrata**: il modello a goroutine e canali rende semplice la programmazione parallela.
- **Libreria standard estesa**: fornisce strumenti per rete, I/O, parsing, testing, sicurezza e molto altro.
- **Compilazione rapida**: i tempi di build sono molto contenuti rispetto ad altri linguaggi compilati.
- **Ecosistema in espansione**: supportato da Google, grandi aziende e una comunità attiva di sviluppatori.

Aziende che adottano Go

Go è impiegato da numerose realtà internazionali di rilievo, tra cui:
Google, **Mercado Libre**, **Bitly**, **American Express**, **Capital One**, **Meta**, **Microsoft**, **Netflix**, **Riot Games**, **Twitch**, **Cloudflare**, **Salesforce**, **Uber**, e molte altre.

Filosofia di progettazione

Go nasce per risolvere problematiche comuni nello sviluppo di software su larga scala:

- tempi di compilazione eccessivi,
- complessità sintattica,
- difficoltà di gestione del parallelismo,
- integrazione debole tra strumenti e ambiente di sviluppo.

I suoi creatori (Robert Griesemer, Rob Pike e Ken Thompson) hanno definito tre principi guida:

1. **Semplicità** → codice leggibile, minimale e coerente.
2. **Efficienza** → prestazioni vicine a C/C++ con gestione della memoria automatica.
3. **Concorrenza naturale** → concetto di goroutine come unità di esecuzione leggera.

Il Go Playground

Il Go Playground è un ambiente online ufficiale per scrivere, compilare ed eseguire codice Go senza installare nulla in locale.

Esempio introduttivo:

```
// Esempio base eseguibile su https://go.dev/play/
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

Il programma definisce un pacchetto `main`, importa il pacchetto `fmt` (formattazione) e invoca la funzione `fmt.Println`, che stampa una stringa su standard output.

Struttura generale di un programma Go

Un file sorgente Go segue una struttura precisa:

```
package main           // dichiarazione del pacchetto
import "fmt"           // importazione delle librerie necessarie
func main() {          // punto di ingresso del programma
    fmt.Println("Esempio base")
}
```

Spiegazione

- `package main` indica che il file fa parte del pacchetto principale, necessario per generare un eseguibile.
- `import "fmt"` importa un pacchetto della libreria standard.
- `func main()` è la funzione di ingresso: ogni eseguibile in Go deve definirla.

Concetto di pacchetto (Package)

In Go, ogni funzione appartiene a un **pacchetto**, che rappresenta un'unità logica di organizzazione del codice.

Regole fondamentali

- Tutti i file appartenenti allo stesso pacchetto devono trovarsi nella **stessa directory**.
- Il nome del pacchetto è dichiarato **all'inizio del file** con la parola chiave `package`.
- I pacchetti possono essere **importati** in altri file tramite la direttiva `import`.

Esempio:

```
package main
import (
    "fmt"
    "math/rand"
)
func main() {
    fmt.Println("Numero casuale:", rand.Intn(10))
}
```

In questo esempio:

- `fmt` fornisce funzioni per la formattazione dell'output.
- `math/rand` offre generatori di numeri casuali.
- Le funzioni o variabili esportate da un pacchetto iniziano sempre con **una lettera maiuscola**.

Regole di visibilità e esportazione

In Go non esistono modificatori di accesso come `public` o `private`.
La **visibilità** di una funzione, variabile o tipo è determinata dalla **lettera iniziale del nome**:

- **Maiuscola** → elemento **esportato** (visibile all'esterno del pacchetto).
- **Minuscola** → elemento **non esportato** (visibile solo all'interno del pacchetto).

Esempio:

```
// Esportata
func PrintMessage() { ... }
// Non esportata
func printMessage() { ... }
```

Esecuzione tramite tool ufficiali

Go fornisce una toolchain integrata per gestire l'intero ciclo di sviluppo:

Comando	Descrizione
`go run file.go`	Compila ed esegue il file specificato
`go build`	Compila il progetto e genera un eseguibile
`go install`	Installa il binario nel percorso predefinito
`go test`	Esegue i test presenti nei file `_test.go`
`go fmt`	Riformatta automaticamente il codice secondo lo stile standard
`go vet`	Analizza il codice e segnala costrutti sospetti o potenziali errori

Struttura tipica di un progetto Go

```
myproject/
│
├── go.mod        # File di definizione del modulo
├── main.go       # Punto di ingresso del programma
└── pkg/          # Pacchetti interni (riutilizzabili)
    └── utils.go
```

Il file `go.mod` definisce il modulo e le dipendenze del progetto.
Un modulo è la **unità di distribuzione e versione** in Go (introdotta dalla versione 1.11).

Inizializzazione di un modulo

`go mod init example.com/myproject go mod tidy`

- `go mod init` crea un nuovo file `go.mod` nella directory corrente e imposta il percorso del modulo.
- `go mod tidy` aggiunge automaticamente le dipendenze mancanti e rimuove quelle non utilizzate.

Compilazione ed esecuzione

- **Compilazione:**

  ```
  go build .
  ```

  Compila i file Go del pacchetto principale e genera un binario.
- **Esecuzione rapida:**

  ```
  go run main.go
  ```

  Compila ed esegue il programma senza salvare il binario.

Testing e documentazione

- `go test` esegue i test di unità definiti nei file con suffisso `_test.go`.
- `go test -v` mostra in dettaglio l'esito di ogni test.
- `go doc <package>.<func>` mostra la documentazione ufficiale di un pacchetto o funzione.

Esempio: `go doc fmt.Println`

Formattazione e analisi statica

Go applica una **formattazione canonica** obbligatoria per tutti i progetti.
L'uso di `go fmt` garantisce che tutto il codice segua uno stile uniforme.

```
go fmt
go vet
```

- `go fmt` corregge automaticamente la formattazione.
- `go vet` analizza potenziali errori semantici (es. variabili inutilizzate, conversioni errate, errori di formattazione).

1.13 Gestione delle dipendenze

- `go get <package>` aggiunge un nuovo pacchetto esterno.
- `go get <package>@vX.Y.Z` installa una versione specifica.
- `go mod verify` controlla l'integrità dei moduli scaricati.
- `go clean -modcache` pulisce la cache dei moduli.

Strumenti consigliati

- **Editor di testo:**
  - Visual Studio Code + estensione Go (ufficiale)
  - GoLand (versione educational disponibile)

Variabili, Costanti e Tipizzazione

Dichiarazione di variabili

In Go, tutte le variabili devono essere dichiarate **esplicitamente** prima dell'utilizzo.
Ogni variabile possiede un tipo statico determinato in fase di compilazione, che non può essere modificato successivamente.

Sintassi generale

```
var nome tipo
```

Esempio:

```
var i int
var s string
var flag bool
```

La parola chiave `var` serve a introdurre una nuova variabile, seguita dal suo identificatore e dal tipo.
La variabile assume un **valore zero** coerente con il tipo dichiarato.

Tipo	Valore di default
Numerici	`0`
Booleani	`false`
Stringhe	`""` (stringa vuota)
Puntatori, Slice, Mappe, Funzioni, Interfacce	`nil`

Inizializzazione esplicita

È possibile assegnare un valore al momento della dichiarazione:

```
var x int = 10
var nome string = "Mario"
var attivo bool = true
```

Go permette anche di omettere il tipo quando il valore iniziale è specificato: in questo caso, **il compilatore deduce automaticamente il tipo**.

```
var x = 10        // tipo int dedotto automaticamente
var y = 3.14      // tipo float64
var msg = "Test"  // tipo string
```

Dichiarazione breve (short declaration)

La sintassi compatta `:=` permette di dichiarare e inizializzare una variabile in un'unica istruzione, lasciando al compilatore l'inferenza del tipo.

```
x := 42
s := "ciao"
b := true
```

Regole:

- `:=` può essere utilizzato **solo all'interno di funzioni**.
- È equivalente a una dichiarazione con `var`, ma senza specificare il tipo.
- **Non può essere usato** per dichiarare costanti.

Costanti

Le costanti rappresentano valori **immutabili** conosciuti in fase di compilazione.
Si dichiarano con la parola chiave `const`.

```
const Pi = 3.14159
const Nome = "Go"
const Attivo = true
```

Le costanti possono essere **tipizzate** o **non tipizzate** (untyped):

```
const x = 10       // untyped int
const y float32 = 3.5 // tipizzata
```

Le costanti non tipizzate assumono un tipo specifico solo quando vengono utilizzate in un contesto che lo richiede.

Esempio: costanti numeriche e conversione implicita

```
const (
    untypedInt   = 1
    untypedFloat = 1.1
)

func needInt(x int) int {
    return x*10 + 1
}

func needFloat(x float64) float64 {
    return x * 0.1
}

func main() {
    fmt.Println(needInt(untypedInt))    // OK → untypedInt diventa int
    fmt.Println(needFloat(untypedInt))  // OK → untypedInt diventa float64
```

```
    fmt.Println(needInt(untypedFloat))  // Errore → tipi incompatibili
}
```

Osservazione

Go effettua **conversioni automatiche** solo per costanti non tipizzate.
Per i valori con tipo esplicito, la conversione deve essere **dichiarata manualmente**.

Conversione di tipo (Casting)

In Go, **non esiste conversione implicita** tra tipi numerici diversi.
È necessario utilizzare un'espressione di conversione esplicita della forma `T(v)`.

```
i := 42
f := float64(i)
u := uint(f)
```

Ogni conversione genera un **nuovo valore**, non modifica l'originale.

2.7 Inferenza di tipo

Quando si dichiara una variabile senza specificare il tipo, Go **deduce automaticamente** il tipo in base al valore assegnato.

Esempio:

```
i := 42           // int
f := 3.142        // float64
g := 0.867 + 0.5i // complex128
```

Note tecniche

- Tutti i valori numerici **letterali** (senza tipo) sono considerati "untyped constants".
- Il compilatore sceglie il tipo più adatto tra quelli disponibili nella categoria numerica.

Tipi di dato di base

Go definisce un insieme ristretto ma completo di **tipi primitivi**.

Categoria	Tipi
Booleani	`bool`
Stringhe	`string`
Interi con segno	`int`, `int8`, `int16`, `int32`, `int64`
Interi senza segno	`uint`, `uint8`, `uint16`, `uint32`, `uint64`, `uintptr`
Alias	`byte` (alias di `uint8`), `rune` (alias di `int32`)
Virgola mobile	`float32`, `float64`
Complessi	`complex64`, `complex128`

Specificità di `int` e `uint`

I tipi `int`, `uint` e `uintptr` **adattano la dimensione al sistema**:

- 32 bit su architetture a 32 bit,
- 64 bit su architetture a 64 bit.

Stringhe e codifica Unicode

Le stringhe in Go rappresentano **sequenze immutabili di byte**, solitamente codificate in **UTF-8**.

Esempio:

```
s := "Ciao, mondo 🌍"
fmt.Println(len(s))                    // lunghezza in byte
fmt.Println(utf8.RuneCountInString(s))  // numero di rune (caratteri)
```

Output:
`Len (Byte): 17 Rune Count: 14`

Rune e byte

- Una **rune** rappresenta un singolo *code point* Unicode (`int32`).
- Un **byte** rappresenta un singolo valore da 0 a 255 (`uint8`).

Intervallo Unicode	Byte per rune (UTF-8)	Esempi
U+0000 – U+007F	1	`A`, `1`, `!`
U+0080 – U+07FF	2	`é`, `ñ`, `£`
U+0800 – U+FFFF	3	`π`, `€`, `한`
U+10000 – U+10FFFF	4	Emoji e simboli speciali

Sintesi della tipizzazione

Caratteristica	Descrizione
Staticità	Il tipo è verificato a compilazione
Esplicitudine	Nessuna conversione automatica implicita
Inferenza	Il compilatore deduce il tipo quando possibile
Sicurezza	Ogni conversione è controllata e garantita dal tipo di destinazione
Coerenza	Ogni variabile mantiene il tipo assegnato per tutta la durata del programma

> **Nota**: La filosofia di Go in materia di tipi è fondata su semplicità e prevedibilità.
> L'assenza di conversioni implicite riduce errori e ambiguità, promuovendo un codice più sicuro e leggibile.

Strutture di Controllo

Le strutture di controllo in Go definiscono il **flusso di esecuzione** di un programma.
Il linguaggio è progettato per offrire un insieme minimale ma potente di costrutti logici, eliminando elementi ridondanti presenti in altri linguaggi (come `while`, `do-while`, o le parentesi obbligatorie di `if` e `for`).

Il ciclo `for`

Go utilizza un **unico tipo di ciclo iterativo**, `for`, che copre tutti i casi d'uso (loop contatore, condizionale, infinito).

Sintassi generale

```
for inizializzazione; condizione; post {
    // corpo del ciclo
}
```

Ogni parte (inizializzazione, condizione, post) è facoltativa.

Esempio classico

```
sum := 0
for i := 0; i < 10; i++ {
    sum += i
}
fmt.Println(sum) // Output: 45
```

- `inizializzazione` → eseguita una sola volta, all'inizio del ciclo;
- `condizione` → valutata prima di ogni iterazione; se false, il ciclo termina;
- `post` → eseguita dopo ogni iterazione.

Versione con parti omesse

È possibile omettere uno o più elementi della dichiarazione:

Solo condizione (stile while)

```
sum := 1
for sum < 1000 {
    sum += sum
}
fmt.Println(sum) // Output: 1024
```

Ciclo infinito

```
for {
    // esecuzione continua
}
```

Per uscire da un ciclo infinito si utilizza l'istruzione `break`.

Uso di break e continue

- `break` → interrompe il ciclo corrente.
- `continue` → termina l'iterazione corrente e passa alla successiva.

Esempio:

```
for i := 0; i < 10; i++ {
    if i%2 == 0 {
        continue // salta i pari
    }
    if i > 7 {
        break // interrompe il ciclo
    }
    fmt.Println(i)
}
```

Output:

```
1 3 5 7
```

L'istruzione `if`

L'istruzione condizionale `if` valuta un'espressione booleana e, se vera, esegue il blocco di codice corrispondente.

Sintassi

```
if condizione {
    // blocco eseguito se vero
}
```

A differenza di C/C++ o Java:

- non sono richieste parentesi tonde `()` intorno alla condizione;
- le parentesi graffe `{}` sono obbligatorie anche per un singolo statement.
  Esempio base

```
x := 5
if x > 0 {
    fmt.Println("x è positivo")
}
```

`if` con clausola else

```
if x < 0 {
    fmt.Println("x è negativo")
} else {
    fmt.Println("x è positivo o nullo")
}
```

Dichiarazione breve nel `if`

Un `if` può contenere un'inizializzazione prima della condizione.
La variabile dichiarata in questa istruzione è visibile solo all'interno del blocco if e dei relativi else.

```
if v := math.Pow(2, 3); v < 10 {
    fmt.Println("v è minore di 10")
} else {
    fmt.Println("v è maggiore o uguale a 10")
}
```

Osservazione
La variabile `v` ***non è accessibile*** al di fuori della struttura condizionale.

else `if` e catene condizionali

Go consente l'uso combinato di più condizioni con else `if`.

```
if x < 0 {
    fmt.Println("negativo")
} else if x == 0 {
    fmt.Println("nullo")
} else {
    fmt.Println("positivo")
}
```

Ogni blocco `else if` viene valutato in ordine sequenziale fino a che una condizione risulta vera.

Lo switch

La struttura `switch` in Go rappresenta una forma compatta e ottimizzata di una catena if - else if.

Sintassi

```
switch espressione {
case valore1:
    // istruzioni
case valore2:
    // istruzioni
default:
    // istruzioni di default
}
```

Caratteristiche distintive

- Ogni `case` viene valutato **in ordine dall'alto verso il basso**.
- Il **primo caso** che corrisponde viene eseguito; gli altri vengono **ignorati**.
- Non è necessario inserire `break` al termine di ogni blocco: in Go è **implicito**.
- I `case` non devono essere costanti, né appartenere a tipi interi.

Esempio pratico:

```go
package main

import (
    "fmt"
    "runtime"
)

func main() {
    switch os := runtime.GOOS; os {
    case "darwin":
        fmt.Println("Sistema operativo: macOS.")
    case "linux":
        fmt.Println("Sistema operativo: Linux.")
    default:
        fmt.Printf("Sistema operativo: %s.\n", os)
    }
}
```

In questo esempio:

- viene utilizzata una *istruzione breve* all'interno del `switch` `(os := runtime.GOOS)`;
- il valore di `os` viene confrontato con diverse stringhe costanti;
- la clausola `default` gestisce i casi rimanenti.

La parola chiave `defer`

La parola chiave `defer` posticipa l'esecuzione di una funzione fino al momento in cui la funzione contenente termina la propria esecuzione.

Esempio:

```go
package main

import "fmt"

func test() {
    defer fmt.Print(" mondo")
    fmt.Print(" crudele")
}

func main() {
    defer fmt.Println("!")
    fmt.Print("Ciao")
    test()
}
```

Output:

```
Ciao crudele mondo!
```

Regole operative

- Gli argomenti della funzione differita vengono valutati immediatamente, ma l'esecuzione della funzione stessa è posticipata.
- Le chiamate `defer` sono gestite secondo il principio LIFO (Last In, First Out): l'ultima differita viene eseguita per prima.

Ambito (Scope) e visibilità

Il linguaggio Go definisce lo scope lessicale in base al blocco {} in cui una variabile è dichiarata.
Una variabile è visibile solo all'interno del blocco in cui è definita e dei sotto-blocchi annidati.

Esempio:

```
x := 10
if x > 5 {
    y := x + 5
    fmt.Println(y) // OK
}
fmt.Println(y) // Errore: y non definita
```

> **Nota**: Go privilegia la chiarezza e la prevedibilità del flusso di controllo.
> Ogni costrutto è progettato per evitare ambiguità sintattiche e garantire comportamenti deterministici, facilitando la verifica formale e la leggibilità del codice.

Costrutto	Caratteristiche principali
`for`	Unico ciclo iterativo; sostituisce `while` e `do-while`
`if`	Struttura condizionale con blocchi obbligatori delimitati da `{}`
`switch`	Alternativa compatta a catene `if-else`, con `break` implicito
`defer`	Posticipa l'esecuzione di una funzione fino al ritorno della funzione chiamante
Scope	Determinato lessicalmente; variabili locali non visibili all'esterno

Tipi Complessi e Strutture Dati

Go fornisce diversi tipi complessi che consentono la gestione efficiente di dati strutturati e dinamici.
Tra questi troviamo: **puntatori**, **struct**, **array**, **slice** e **mappe**.
Ognuno di essi risponde a precise esigenze di modellazione e controllo della memoria.

Puntatori

Un **puntatore** è una variabile che contiene l'indirizzo di memoria di un valore.
In Go, i puntatori offrono un controllo esplicito dei riferimenti senza permettere l'aritmetica sui puntatori, come avviene in C.

Sintassi

```
var p *int   // p è un puntatore a int
i := 42
p = &i       // p contiene l'indirizzo di i
fmt.Println(*p) // Dereferenziazione: legge il valore di i
```

Osservazioni

- Il tipo `*T` rappresenta un puntatore a un valore di tipo `T`.
- L'operatore `&` restituisce l'indirizzo di una variabile.
- L'operatore `*` dereferenzia il puntatore, accedendo o modificando il valore puntato.
- Il valore zero di un puntatore è `nil`.
- Go **non consente l'aritmetica sui puntatori**, prevenendo accessi non sicuri alla memoria.

Esempio completo:

```
package main
import "fmt"
func main() {
    var p *int
    i := 42
    p = &i
    fmt.Println(p)   // indirizzo di i
    fmt.Println(*p)  // valore di i
    *p = 21          // modifica il valore di i tramite puntatore
    fmt.Println(i)   // stampa 21
}
```

Strutture (`struct`)

Una **struct** è un aggregato di campi eterogenei, utilizzato per definire nuovi tipi di dati complessi.

Definizione e dichiarazione

```
type Vertex struct {
    X, Y int
}
```

Esempi di inizializzazione

```
var (
    v1 = Vertex{1, 2}       // inizializzazione per posizione
    v2 = Vertex{X: 1}       // inizializzazione per nome (Y = 0 implicito)
    v3 = Vertex{}           // valori zero (X = 0, Y = 0)
    p  = &Vertex{1, 2}      // puntatore a struct
)
```

Accesso ai campi

```
fmt.Println(v1.X)  // accesso diretto al campo
p.Y = 10           // modifica del campo tramite puntatore
```

Osservazioni:

- I campi sono accessibili tramite la **dot notation** (`variabile.campo`).
- L'accesso tramite puntatore (`(*p).campo`) può essere abbreviato in `p.campo`.
- Le struct non supportano l'ereditarietà, ma si possono **comporre** altre struct come campi.

Struct anonime

Le struct possono essere dichiarate senza un nome di tipo, utili per dichiarazioni locali o temporanee.

```
a := struct
{
    i int
    b bool
}{1, true}
fmt.Println(a.i, a.b)
```

> L'uso di struct anonime è comune in test o funzioni dove non serve riutilizzare il tipo.

Array

Un **array** in Go è una sequenza numerata di elementi dello stesso tipo e **di dimensione fissa**.

Dichiarazione e inizializzazione

```
var a [10]int                   // array di 10 interi (tutti 0)
primes := [6]int{2, 3, 5, 7, 11, 13}
```

Proprietà fondamentali

- La **lunghezza** di un array è parte del suo tipo: `[6]int` e `[5]int` sono tipi diversi.
- Gli array **non sono ridimensionabili**.
- I valori non inizializzati assumono il **valore zero** del tipo.
  Esempio:

```
V := [6]int{2, 3, 5, 7}
fmt.Println(V) // [2 3 5 7 0 0]
```

Slice

Una **slice** è una vista dinamica su una parte di un array.
A differenza degli array, le slice hanno **dimensione variabile** e rappresentano un livello di astrazione più flessibile.

Sintassi

```
a := [6]int{2, 3, 5, 7, 11, 13}
s := a[1:4]  // slice che include elementi da indice 1 a 3
fmt.Println(s) // [3 5 7]
```

Caratteristiche

- Le slice **non memorizzano dati direttamente**, ma fanno riferimento a un array sottostante.
- La modifica di una slice influisce sull'array originario e su altre slice che lo condividono.
- La **lunghezza** (`len`) e la **capacità** (`cap`) di una slice sono misurabili tramite funzioni integrate.

```
fmt.Println(len(s)) // numero di elementi nella slice
fmt.Println(cap(s)) // numero di elementi disponibili nell'array sottostante
```

Letterali Slice

I letterali slice creano implicitamente un array e una slice che lo referenzia.

```
s := []bool{true, true, false}
fmt.Println(s)
```

Il valore zero di una slice è `nil`.

Funzione `make`

La funzione integrata `make` alloca un array e restituisce una slice che lo referenzia.

```
a := make([]int, 5)         // len(a) = 5, cap(a) = 5 → [0 0 0 0 0]
b := make([]int, 0, 5)      // len(b) = 0, cap(b) = 5
```

Se la capacità è maggiore della lunghezza, la slice può essere estesa senza riallocazione.

Estensione di una Slice

È possibile estendere una slice verso destra, **ma non verso sinistra**.

```
s := []int{2, 3, 5, 7, 11, 13}
s = s[:0]  // len=0, cap=6
s = s[:4]  // len=4, cap=6
```

> ⚠ Non è possibile estendere una slice oltre la sua capacità.

Append

La funzione `append` aggiunge uno o più elementi in coda a una slice.
Se la capacità è esaurita, viene allocato un nuovo array.

```
var s []int
s = append(s, 1, 2, 3)
fmt.Println(s) // [1 2 3]
```

Caso di studio

```
s := []int{9000}[:0]  // slice con len=0, cap=1
s = append(s, 0)
fmt.Println(len(s), cap(s), s) // 1 1 [0]
```

Range

L'istruzione `range` itera su slice, array, stringhe e mappe. Restituisce due valori per iterazione: **indice** e **valore**.

```
pow := []int{1, 2, 4, 8}
for i, v := range pow {
    fmt.Printf("2**%d = %d\n", i, v)
}
```

Mappe

Le **mappe** rappresentano collezioni non ordinate di coppie chiave/valore.
Sono implementate come tabelle hash.

Dichiarazione e inizializzazione

```
var m map[string]string     // m è nil, non utilizzabile
m = make(map[string]string) // inizializzazione
m["Bell Labs"] = "Cooked"
fmt.Println(m["Bell Labs"])
```

Letterali mappa

```
type Vertex struct {
    Lat, Long float64
}

var m = map[string]Vertex{
    "Bell Labs": {40.68433, -74.39967},
    "Google":    {37.42202, -122.08408},
}
```

Operazioni fondamentali

```
m["Key"] = "Valore"         // inserimento o aggiornamento
val := m["Key"]             // lettura
delete(m, "Key")            // cancellazione
```

Verifica presenza di una chiave

```
val, ok := m["Key"]
if ok {
    fmt.Println("Presente:", val)
} else {
    fmt.Println("Assente")
}
```

4.12 Considerazioni finali

Tipo	Caratteristiche principali
Puntatore	Riferimento a un valore; dereferenziazione con `*`; valore zero `nil`
Struct	Aggregato di campi; accesso tramite `.`; supporta composizione
Array	Dimensione fissa; tipo determinato anche dalla lunghezza
Slice	Vista dinamica su un array; dimensione variabile; creato con `make` o letterale
Mappa	Collezione chiave/valore; valore zero `nil`; accesso sicuro con `ok`

> 💡 **Nota**: I tipi complessi di Go sono progettati per garantire sicurezza e prevedibilità, evitando errori comuni nella gestione della memoria e semplificando l'interoperabilità tra strutture statiche e dinamiche

Funzioni e Chiusure (Closures)

Le funzioni in Go sono **unità fondamentali di astrazione** e rappresentano il meccanismo principale per la strutturazione logica del codice.
Go supporta **funzioni di primo ordine**, ovvero funzioni che possono essere assegnate a variabili, passate come argomenti o restituite come valori.

Definizione di funzione

Una funzione è definita tramite la parola chiave `func`, seguita da:

1. il **nome della funzione** (facoltativo, per funzioni anonime),
2. la **lista dei parametri**,
3. il **tipo di ritorno** (opzionale),
4. il **corpo della funzione**, racchiuso tra parentesi graffe.

Esempio base

```go
func somma(a int, b int) int {
    return a + b
}

func main() {
    risultato := somma(3, 4)
    fmt.Println(risultato) // Output: 7
}
```

Osservazioni

- Le funzioni in Go **devono specificare il tipo di ogni parametro**, salvo che più parametri consecutivi condividano lo stesso tipo.
  Esempio: `func somma(a, b int)` è equivalente a `func somma(a int, b int)`.
- Ogni funzione **restituisce zero o più valori**.
- L'ordine dei parametri è **significativo** e non supporta argomenti nominati.

Funzioni con più valori di ritorno

Go permette a una funzione di restituire più valori contemporaneamente.
Questo approccio è molto usato per restituire un risultato e un eventuale errore.

```go
func dividi(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("divisione per zero")
    }
    return a / b, nil
}

func main() {
    risultato, err := dividi(10, 2)
    if err != nil {
        fmt.Println("Errore:", err)
        return
    }
    fmt.Println("Risultato:", risultato)
}
```

Osservazione:
Il valore `nil` rappresenta l'assenza di errore.
Questo pattern (`valore, err`) è considerato **idiomatico** in Go.

Valori funzione

In Go, le funzioni possono essere trattate come **valori di prima classe**.
Ciò significa che possono essere assegnate a variabili, passate come parametri o restituite da altre funzioni.
Esempio:

```go
func hypot(x, y float64) float64 {
    return math.Sqrt(x*x + y*y)
}

func compute(fn func(float64, float64) float64) float64 {
    return fn(3, 4)
}

func main() {
    fmt.Println(compute(hypot))     // Stampa 5
    fmt.Println(compute(math.Pow))  // Stampa 81
}
```

Analisi:

- `fn` è un parametro di tipo **funzione**, che accetta due `float64` e restituisce un `float64`.
- La funzione `compute` può accettare **qualsiasi funzione** compatibile con questa firma.

> Questo approccio è alla base del **funzionamento dei callback** e delle **funzioni di ordine superiore**.

Funzioni anonime

Una funzione può essere definita **senza nome**, diventando un valore anonimo.
È utile per definire comportamenti inline o temporanei.

```
    return a + b
}
fmt.Println(f(2, 3)) // Output: 5
Le funzioni anonime possono essere **eseguite immediatamente** (IIFE – _Immediately Invoked
Function Expression_):
result := func(x int) int {
    return x * x
}(5)
fmt.Println(result) // Output: 25
```

Chiusure (Closures)

Una **closure** è una funzione che **cattura** e utilizza variabili esterne al proprio corpo lessicale.
Ogni istanza di closure mantiene il proprio **stato indipendente** delle variabili catturate.

Esempio classico:

```go
func adder() func(int) int {
    sum := 0
    return func(x int) int {
        sum += x
        return sum
    }
}

func main() {
```

```go
    pos, neg := adder(), adder()
    for i := 0; i < 5; i++ {
        fmt.Println(pos(i), neg(-2*i))
    }
}
```

Output:

```
0 0
1 -2
3 -6
6 -12
10 -20
```

Spiegazione

- La funzione `adder()` restituisce una funzione anonima che mantiene una variabile `sum` nel proprio contesto.
- Ogni chiamata a `adder()` crea una **nuova istanza** della closure, con la propria `sum` indipendente.
- L'ambiente lessicale (`sum`) rimane attivo anche dopo che `adder()` ha terminato l'esecuzione.

> ⚙️ **Le closure implementano il concetto di stato interno persistente** all'interno di funzioni di ordine superiore.

Funzioni variadiche

Le **funzioni variadiche** accettano un numero variabile di argomenti dello stesso tipo.
Si definiscono con la notazione `...` davanti al tipo del parametro.

```go
func somma(numeri ...int) int {
    totale := 0
    for _, n := range numeri {
        totale += n
    }
    return totale
}

func main() {
    fmt.Println(somma(1, 2, 3))        // 6
    fmt.Println(somma(10, 20, 30, 40)) // 100
}
```

Osservazioni

- Il parametro variadico è trattato come una **slice** all'interno della funzione.
- È possibile passare una slice esistente usando la notazione `f(slice...)`.

Defer e chiusura di risorse

Il costrutto `defer` è spesso utilizzato insieme a funzioni anonime per **rinviare la chiusura di risorse** fino al termine dell'esecuzione della funzione corrente.

Esempio:

```go
func leggiFile(nome string) {
    f, err := os.Open(nome)
    if err != nil {
        log.Fatal(err)
    }
    defer f.Close() // chiusura automatica del file al termine

    // elaborazione del file
}
```

> **Buona pratica**: utilizzare `defer` immediatamente dopo l'apertura di una risorsa garantisce la chiusura sicura anche in caso di errori o panic.

Considerazioni generali

Caratteristica	Descrizione
Funzione di primo ordine	Può essere assegnata, passata o restituita
Funzione anonima	Non ha nome, utile per definizioni inline
Closure	Cattura variabili esterne e mantiene stato persistente
Funzione variadica	Accetta numero variabile di argomenti
Defer	Posticipa l'esecuzione, utile per la gestione delle risorse

> **Nota**: Le funzioni in Go sono progettate per favorire la chiarezza e l'incapsulamento dello stato. Le chiusure, unite al tipaggio statico, offrono un potente equilibrio tra sicurezza e flessibilità operativa.

Gestione dei Moduli e Toolchain di Go

La **toolchain di Go** fornisce un insieme integrato di strumenti per la gestione dei moduli, la compilazione, il testing e l'analisi statica del codice.
Il suo design mira a garantire **riproducibilità, semplicità e trasparenza** nello sviluppo software.

Concetto di modulo (`go mod`)

A partire da Go 1.11, il linguaggio adotta un sistema nativo per la **gestione delle dipendenze** basato sui *moduli*.
Un modulo Go rappresenta un **insieme coerente di pacchetti** versionati insieme e identificati da un percorso univoco.

Obiettivi principali del sistema a moduli

- Tracciare versioni e dipendenze in modo esplicito.
- Garantire build riproducibili.
- Eliminare la necessità di strumenti esterni (come dep o vendoring manuale).

Inizializzazione di un modulo

Il comando `go mod init` serve per creare un nuovo modulo all'interno della directory corrente.
Esso genera un file `go.mod` che definisce:

- il **nome del modulo** (che coincide con il percorso importabile),
- le **dipendenze** e le relative versioni,
- i **vincoli** di compatibilità semantica (semver).

Esempio

```
mkdir myproject && cd myproject
go mod init example.com/myproject
```

Il file `go.mod` risultante sarà simile al seguente:

```
module example.com/myproject  go 1.22
```

Comandi principali per la gestione dei moduli

Comando	Descrizione
`go mod init <path>`	Inizializza un nuovo modulo
`go mod tidy`	Aggiorna automaticamente le dipendenze (aggiunge mancanti, rimuove inutili)
`go mod download`	Scarica tutti i moduli richiesti nella cache locale
`go mod verify`	Verifica che i moduli scaricati corrispondano agli hash in `go.sum`
`go mod vendor`	Crea una cartella `vendor/` con copie locali delle dipendenze
`go list -m all`	Elenca tutte le dipendenze del modulo, incluse quelle transitive

File di controllo: `go.mod` e `go.sum

- `go.mod`: definisce la versione minima richiesta di ogni dipendenza.
- `go.sum`: contiene gli *hash crittografici* dei moduli importati, per garantire l'integrità dei download.
  Esempio di `go.sum`:

```
golang.org/x/text v0.9.0 h1:YF/v3Yv...
golang.org/x/text v0.9.0/go.mod h1:XRxV...
```

> Questi hash assicurano che il codice scaricato non sia stato alterato.

Struttura standard di un progetto Go

L'organizzazione tipica dei progetti Go segue una convenzione chiara e coerente:

```
myproject/
│
├── go.mod            # definizione del modulo e dipendenze
├── go.sum            # hash di verifica delle dipendenze
├── main.go           # punto di ingresso (package main)
└── pkg/              # pacchetti riutilizzabili
    ├── utils.go
    └── handler.go
```

Convenzioni

- Il **pacchetto** `main` rappresenta il punto di ingresso per l'eseguibile.
- I pacchetti interni (`pkg/`, `internal/`, ecc.) contengono codice riutilizzabile.
- Ogni directory rappresenta un **pacchetto autonomo**.

Esecuzione e compilazione

Esecuzione diretta

```
go run main.go
```

Compila ed esegue il sorgente specificato senza creare un binario permanente.

Compilazione

```
go build .
```

Compila il pacchetto corrente (se contiene `main`) e genera un eseguibile nella directory corrente.

Build con nome personalizzato

```
go build -o app_name
```

Crea un binario denominato `app_name`.

Installazione globale

```
go install
```

Compila e installa l'eseguibile nella cartella binaria predefinita di Go (`$GOBIN` o `$GOPATH/bin`).

Testing

Go include un **framework di testing integrato** nella toolchain, basato su file denominati con suffisso `_test.go`.

Comandi fondamentali

Comando	Descrizione
`go test`	Esegue i test nella directory corrente
`go test ./...`	Esegue tutti i test in modo ricorsivo
`go test -v`	Esegue i test in modalità verbosa
`go test -cover`	Calcola la copertura del codice

Esempio di test:

```go
package mathutil
import "testing"
func TestSomma(t *testing.T) {
    risultato := Somma(2, 3)
    if risultato != 5 {
        t.Errorf("Somma(2,3) = %d; atteso 5", risultato)
    }
}
```

Formattazione e analisi statica

Go impone una **formattazione standardizzata** e automatica del codice.

Formattazione

`go fmt`
Riformatta tutti i file Go nella directory corrente secondo lo stile ufficiale.

Analisi statica

`go vet`
Esegue controlli semantici avanzati su:

- variabili non utilizzate,
- assegnazioni errate,
- formattazioni incompatibili,
- uso improprio di `Printf`, `range`, ecc.

> ⚙️ L'uso combinato di `go fmt` e `go vet` è considerato obbligatorio nei flussi di revisione del codice.

Documentazione

Go fornisce il comando `go doc` per accedere alla documentazione integrata dei pacchetti.
Esempio:

```
go doc fmt.Println
```

Questo comando visualizza la documentazione ufficiale della funzione `fmt.Println` estratta direttamente dal sorgente.

Gestione delle dipendenze esterne

Comando	Descrizione
`go get <package>`	Aggiunge una nuova dipendenza o aggiorna una esistente
`go get <package>@v1.2.3`	Installa una versione specifica
`go get -u ./...`	Aggiorna tutte le dipendenze alle versioni più recenti
`go get -u -patch ./...`	Aggiorna solo alle versioni patch più recenti

> Le versioni dei moduli seguono la **Semantic Versioning (SemVer)**: `MAJOR.MINOR.PATCH`.

Pulizia e manutenzione dell'ambiente

Rimozione cache moduli

```
go clean -modcache
```

Elimina la cache locale dei moduli (`$GOPATH/pkg/mod`), forzando il loro riscaricamento al successivo build.

Pulizia completa

```
go clean
```

Rimuove i file binari e temporanei generati dalla compilazione.

Verifica e integrità dei moduli

Go garantisce la sicurezza delle dipendenze tramite **verifica crittografica**.

```
go mod verify
```

Questo comando controlla che gli hash dei moduli scaricati corrispondano ai valori registrati nel file `go.sum`.
In caso di discrepanze, la build viene interrotta.

Ambiente e configurazione

Il comando `go env` mostra le variabili d'ambiente attualmente configurate.

```
go version
go env
```

Variabili principali

Variabile	Descrizione
`GOROOT`	Percorso di installazione dell'ambiente Go
`GOPATH`	Directory che contiene `src`, `pkg` e `bin`
`GOBIN`	Percorso di installazione dei binari eseguibili
`GOOS` / `GOARCH`	Sistema operativo e architettura di compilazione

> **Nota**: L'ecosistema Go privilegia la coerenza e la prevedibilità.
> La toolchain è pensata per ridurre la complessità, assicurando ambienti di compilazione identici e flussi di sviluppo ripetibili e tracciabili.